

◆ FULL-STACK ENGINEER · SEOUL

문제를 읽고, 구조로 답합니다.

비즈니스 도메인을 시스템 구조로 풀어내고,
기획부터 DevOps까지 제품의 전 과정을 책임지는 김어진입니다.

Email

uhjinkim9@gmail.com

GitHub

https://github.com/uhjinkim9

Portfolio

https://portfolio.mercury-lab.uk

◆ KEY COMPETENCIES

끝까지 만드는 개발자의 세 가지 힘



도메인 분석 & 구조 설계

현업의 페인 포인트를 분석해
비즈니스 규칙을 데이터 구조로 정제하고,
운영 가능한 아키텍처를 설계합니다.



주도적 실행력

UI/UX 설계부터 API, 배포 자동화와 가옴 환경까지
제품 구축의 전 과정을 완주합니다.



빠른 기술 적응 & AI 활용

생소한 언어나 스택을 빠르게 습득하며,
AI 도구를 적극 활용해 개발 생산성을 극대화합니다.

◆ ENTERPRISE PLATFORM · 2025.03 — 2026.08

사내 그룹웨어 플랫폼

MSA 기반 그룹웨어 플랫폼 설계부터 배포까지

- Next.js
- NestJS
- TypeORM
- MariaDB
- RabbitMQ
- Kubernetes
- Jenkins

역할 | 기획 · 아키텍처 · 풀스택 · DevOps

규모 | 2-3명

분산된 협업과 업무 관리 기능을 하나의 서비스로 통합하도록 설계한 MSA 기반 그룹웨어 플랫폼입니다.

메일·전자결재·게시판 등 기본 기능에 근태·연차 프로세스를 연동하고,
필요한 솔루션의 공백에는 직접 MVP를 구축하며 구조 설계부터 배포까지 전 과정을 주도했습니다.

메일

전자결재

문서

커뮤니티

일정 및 근태

관리자

사내 시스템

UCUBERS

🔍

👤

💬

💡

🔗

출근

퇴근

출근 전

☰

🔔

↔

⋮

오늘의 태스크가 없습니다.

July 2026

<

>

📅 일정

↔

⋮

S	M	T	W	T	F	S
			1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30	31	

일정 목록이 없습니다.

✉ 메일

↔

⋮

서지유

Re: Stage 환경 Mailu 메일 서버 배포를 위한 인프라 권한 요청

2026. 7. 29. 오후 1:37

📁 결재

↔

⋮

내 승인 대기

내 기안

참조

7월 29일 연차

김어진

결재 중

2026-07-29

📢 공지

↔

⋮

[매뉴얼] 직장 건강검진 휴가 사용 안내

2026. 4. 24.

[매뉴얼] 전사 그룹웨어 서비스 전환 일정 변경 안내

2026. 4. 24.

[일반] 공지알림

2026. 7. 24.

[일반] 테스트

2026. 4. 27.

[매뉴얼] 링크드인 채널 개설 안내

2026. 4. 24.

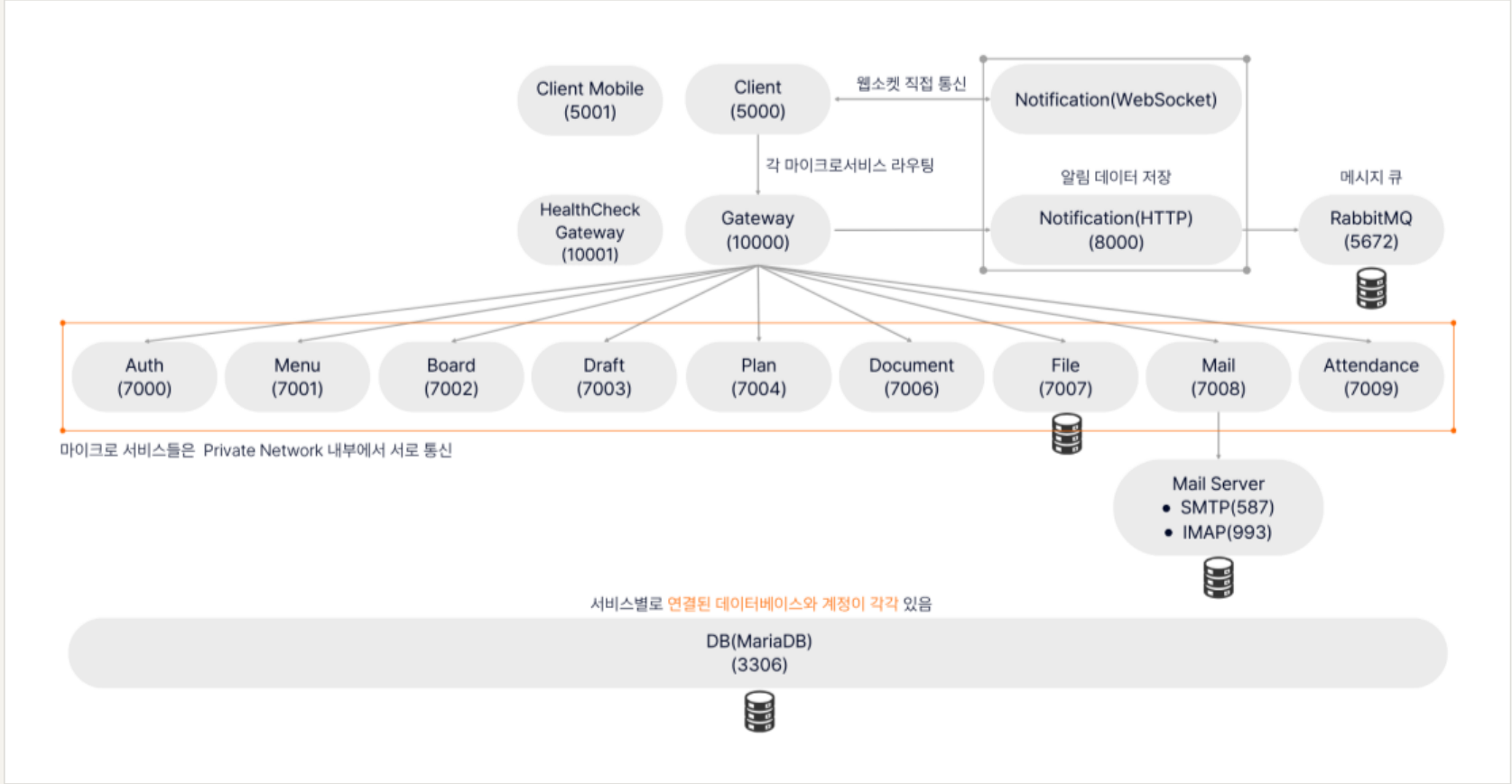
🏠

◆ CONTEXT

확장 가능한 그룹웨어를 처음부터 설계하다

메일, 게시판, 전자결재, 캘린더, 근태 등 서로 다른 업무 도메인을 하나의 플랫폼 안에서 일관되게 제공해야 했습니다.
기업별 데이터와 권한이 분리되는 멀티 테넌시 구조를 기반으로, 개발자에게는 찾기 쉬운 코드 구조를, 사용자에게는 학습이 적은 인터페이스를 목표로 삼았습니다.

- ✓ 기획·설계·개발 리드 — 타사 서비스 분석하여 화면·기능 설계
- ✓ Next.js, NestJS / API Gateway 중심 MSA 구조
- ✓ React Native 모바일 앱 AI 활용하여 0→1 구축



아키텍처



초기 홈 대시보드 화면 기획

◆ PROBLEM 1

이벤트 처리 모듈과 발행 모듈을 분리하다

초기 설계 시 낮은 결합도를 목표로 11개의 마이크로서비스를 구성했지만, 개발 중 OutOfMemory와 CPU 사용량 증가로 기동이 멈춰버리는 현상이 발생했습니다.

자원 모니터링과 로그를 확인한 뒤 유사 도메인 한 쌍을 통합하여 11 → 10개로 서비스 수를 조정했습니다. 서비스 구성과 DB도 함께 통합 및 리팩토링하여 개발 환경의 자원 부담을 낮추었습니다.

낮은 결합도가 이론상 이상적이지만, 현실적으로 활용 가능한 자원의 한계를 함께 고려해야 한다는 점을 깨달았습니다.

- ✓ 유사 도메인 통합과 서비스 경계 재정의
- ✓ 서비스 구성 및 DB 커넥션 구조 점검·리팩토링
- ✓ 개발 환경의 메모리 부담과 응답 지연 완화

서비스 수 11 → 10 조정하여 개발 환경 메모리 절감

◆ PROBLEM 2

메인 요청과 후처리 이벤트의 생명주기를 분리하다

메인 기능 트랜잭션에 푸시 알림과 메일 발송 로직이 동기적으로 묶여 있어 푸시 알림 연동 지연이나 실패가 핵심 기능의 오류로 전파되었고, 긴 발송 로직을 함수마다 재작성해야 하는 불필요한 코드 중복이 발생했습니다.

따라서 발행과 처리를 분리할 필요성을 느껴, 메시징을 중계하는 메시지큐 도입을 고려했습니다. Kafka와 RabbitMQ를 비교한 뒤 대규모 스트리밍보다 이벤트 라우팅이 중요한 프로젝트 특성에 맞춰 RabbitMQ를 선택했습니다.

각 서비스에 이벤트 발행 모듈을 생성하고, Notification 서비스를 구성하여 발행된 이벤트 처리 담당 모듈로 분리했습니다.

- ✓ 이벤트 발행 모듈과 처리 모듈 분리
- ✓ 메인 API를 막지 않는 비동기 후처리
- ✓ 팀 공통 이벤트 발행 규칙과 재사용 구조 정립

발행 모듈과 처리 모듈을 분리하여 메인 요청과 독립된 비동기 후처리 환경 구축

◆ DOMAIN

인사 담당자와 협업해 근태·연차 프로세스를 자동 연동하다

기본 그룹웨어 기능인 메일·전자결재·게시판에 더해, 근태와 연차 업무를 하나의 플랫폼에서 처리할 수 있도록 인사 담당자와 협업했습니다.

현업 피드백을 바탕으로 연차·반차 반영 방식, 연차 차감 기준과 근태 인정 범위 등의 기본 규칙 및 예외 조건을 정리했습니다.
예외 조건까지 반영할 수 있도록 정책 생성 프로세스를 시스템화하고,
프로세스에 따라 근태·연차 데이터가 그룹웨어에 자동으로 반영되도록 구현했습니다.

- ✓ 인사 담당자 피드백 기반 반영 가능한 요구사항 도출
- ✓ 연차·반차 반영 및 근태 인정 범위의 기본·예외 규칙 정리
- ✓ 복잡한 예외 조건을 흡수하는 정책 생성 프로세스 및 비즈니스 로직 설계
- ✓ 그룹웨어 자동 연동을 통한 인사 담당자의 반복 수동 업무 축소

인사 업무 도메인을 분석해 예외 정책을 정리하고 근태·연차 프로세스 자동화 달성

◆ 0 TO 1

외부 솔루션의 공백을 AI 활용 메신저 MVP로 대응하다

외부 메신저 솔루션을 도입하기 어려워지면서 사내 실시간 소통 기능을 별도로 마련해야 하는 상황이 발생했습니다.

기존 그룹웨어에서 메신저 화면을 연결할 위치와 사용 흐름을 먼저 정하고, 1:1 채팅·파일 첨부·클립보드 이미지 붙여넣기를 MVP 범위로 정의했습니다.

정리한 요구사항을 바탕으로 AI 도구를 활용해 Node.js와 WebSocket 기반 기능을 구현했으며, 3일 안에 필요한 범위의 MVP를 완성했습니다.

- ✓ 그룹웨어 내 메신저 화면 위치와 사용 흐름 설계
- ✓ 1:1 채팅·파일 첨부·클립보드 이미지 붙여넣기 구현
- ✓ Node.js 및 WebSocket 기반의 실시간 메신저 MVP 개발
- ✓ AI 도구를 적극 활용하여 3일 만에 요구사항 구현

AI 도구 활용을 통한 실시간 메신저 MVP 0 to 1 신규 구축

◆ COWORK

AI를 쓰는 팀의 코드 정합성을 문서로 지키다

AI 도구 활용이 늘면서 팀원마다 다른 코드 패턴과 중복 함수·컴포넌트가 생성되기 시작했습니다.

폴더 구조, 컴포넌트 재사용 기준, API 호출 패턴과 커밋 규칙을 주제별 Markdown 문서로 명문화했습니다.
각 주제를 대표하는 인덱스 파일에는 관련 문서의 경로와 참조 순서를 정리해, 필요한 규칙을 단계적으로 찾아갈 수 있도록 구성했습니다.

프롬프트에는 작업과 관련된 인덱스 번호를 함께 전달해 AI가 해당 목차를 따라 필요한 문서만 참조하도록 했습니다.
모든 가이드를 한 번에 컨텍스트로 제공하지 않아도 되어 불필요한 문서 로딩을 줄이고 토큰 사용 효율을 높이는 데 기여했습니다.

- ✓ 주제별 가이드 및 문서 경로를 체계화한 인덱스 구조 구축
- ✓ 프롬프트 내 인덱스 기반 선택적 문서 참조 환경 구현
- ✓ 코드 패턴 일관성 확보 및 불필요한 코드·컴포넌트 중복 억제
- ✓ 컨텍스트 최적화를 통한 AI 토큰 사용 효율 및 응답 정확도 개선

AI 컨텍스트 참조 가이드 표준화로 코드 정합성 향상 및 토큰 사용량 절감 기여

◆ DELIVERY

인프라팀 가이드를 바탕으로 GitOps 배포와 가오픈까지

개발이 완료된 기능이 검증 단계를 거쳐 가오픈 환경에 안정적으로 반영될 수 있도록 GitHub 브랜치 흐름을 정리했습니다.

방화벽 등 기반 인프라 설정은 인프라팀에 요청하고, 제공받은 가이드에 따라 Helm 차트 기반 GitOps 배포 구성을 작성했습니다.
첫 GitOps 배포 경험이었던 만큼 AI 도구를 검증 및 오류 디버깅 보조로 적극 활용했으며,
development에서 기능을 통합하고 stage에서 검증한 뒤 production으로 반영하는 흐름을 구성해 Kubernetes 환경의 가오픈 배포까지 완료했습니다.

브랜치 merge 시 Jenkins 파이프라인을 타고 빌드 및 이미지 푸쉬 후 ArgoCD를 통해 배포되는 흐름을 이해하고 시스템에 적용했습니다.

- ✓ 방화벽 등 기반 인프라 설정을 인프라팀과 협업
- ✓ 인프라팀 가이드에 따른 Helm 기반 GitOps 배포 구성
- ✓ development → stage → production 브랜치 전략 정립
- ✓ AI 기반 오류 검증을 통한 Kubernetes 가오픈 배포 완수

AI 도구를 활용해 생소한 GitOps·Helm 배포 환경을 조기에 습득하고 성공적으로 가오픈 배포 완성

◆ SOLO ENTERPRISE · 2024.09 — 2025.02

사내 자산 관리 시스템

엑셀 프로세스를 자산 중심 시스템으로

Java

전자정부프레임워크

React

MariaDB

MyBatis

역할

요구 분석 · 데이터 모델링 · API · 운영

규모

1명

입사 후 가장 먼저 맡은 업무입니다.

프로젝트별 엑셀에 흩어진 자산 정보를 하나의 시스템으로 통합했습니다.

현업 인터뷰부터 데이터 모델링, API 구현까지 단독 수행하며 업무 언어를 추적 가능한 자산 생애주기로 변환했습니다.

ERP

마이페이지로그아웃

인사

총무

인사현황

자산 관리

자산 현황

프로젝트별 자산 현황

유허 자산 현황

계약 관리

프로젝트 근무지

사용 가이드

관리자

시스템관리

Home / 총무 / 자산 관리

자산 관리

관리번호

자산구분

소속부서

사용자

프로젝트코드

카테고리

자산상태

조회

초기화

전체

렌탈

구매

자산 등록

* 신규 자산 등록 시, '상세 다운로드' 파일에 추가 입력 후 '일괄 등록'으로 업로드해주세요.

일괄 등록

일괄 변경

개요 다운로드

☐	No.	-	-	-	관리번호	사번	사용자	소속부서	프로젝트	프로젝트코드	근무지	프로젝트 주소	자산상태	자산구분
☐	846	상세	지급/이관	회수									사용중	
☐	845	상세	지급/이관	회수									사용중	
☐	844	상세	지급/이관	회수									사용중	
☐	843	상세	지급/이관	회수									사용중	
☐	842	상세	지급/이관	회수									사용중	
☐	841	상세	지급/이관	회수									사용중	
☐	840	상세	지급/이관	회수									사용중	
☐	839	상세	지급/이관	회수									사용중	
☐	838	상세	지급/이관	회수									사용중	
☐	837	상세	지급/이관	회수									사용중	
☐	836	상세	지급/이관	회수									사용중	
☐	835	상세	지급/이관	회수									사용중	
☐	834	상세	지급/이관	회수									사용중	
☐	833	상세	지급/이관	회수									사용중	

◆ CONTEXT

엑셀의 행 사이에서 사라지는 자산 이력

보유 자산 현황, 렌탈 자산 현황, 프로젝트별 자산 비용 등 여러 엑셀 파일에 자산을 수기로 나눠 관리하고 있어 총무 담당자가 실제 자산의 이동 이력을 추적하기 어려웠습니다. 누락된 데이터와 파일별로 상이한 작성 기준 때문에 프로젝트별 비용 집계에도 반복적인 확인 작업이 필요했습니다.

- ✓ 자산 이동·유휴 이력 추적 불가
- ✓ 수기 입력 오류와 데이터 누락
- ✓ 프로젝트별 비용 집계 공수 증가

◆ MODELING

관리의 기준을 사용자에서 자산으로 전환하다

사용자가 무엇을 보유했는지 기록하는 방식에서 자산 한 개가 어떤 상태와 위치를 거쳤는지 추적하는 모델로 관점을 전환했습니다. 생성 → 할당 → 회수 → 수리 → 재할당으로 이어지는 생애주기를 정의하고 모든 상태 변화를 자산 고유 키에 연결했습니다.

- ✓ 자산 고유 키 기반 이력 관리
- ✓ 상태 변경과 이동 이력 분리
- ✓ 인사 모듈 연계를 통한 입·퇴사 자산 처리 자동화

GA_ASSET_SEQ 자산 관리번호 시퀀스 테이블
A-Z SEQ_CATEGORY - 기준 카테고리
123 NEXT_SEQ - 해당 기준의 다음 시퀀스 값

GA_ASSET_EXTEND_HISTORY 자산 계약 연장 이력
123 IDX - 인덱스
A-Z AST_KEY - 자산 관리번호 (GA_ASSET.AST_KEY)
A-Z CTR_NO - 계약번호
123 CTR_STATUS_IDX - GA_ASSET_CTR_STATUS.IDX
A-Z PREV_RTN_PLAN_DATE - 연장 전 반납예정일
A-Z NEW_RTN_PLAN_DATE - 연장 후 반납예정일
A-Z EXTEND_DATE - 연장 처리일
A-Z MEMO - 연장 사유
A-Z CREATE_USER - 처리자
🕒 CREATE_DATE - 등록일자

GA_ASSETS_HISTORY 총무-지급이력
123 ID_HISTORY
A-Z AST_KEY - 자산 키
A-Z EMP_NO - 사용자 사번
A-Z DEPT_CD - 사용자 소속부서 코드
A-Z DEPT_NM - 부서명
A-Z PROJECT_CD - 프로젝트 코드
A-Z PROJECT_NM - 프로젝트명
A-Z KOR_NM - 자산 사용 책임자
A-Z AST_USER - 자산 사용자
A-Z PROVIDE_DATE - 지급일
A-Z AST_PROV_STAT - 지급 상태(지급/이관/반납)
🕒 CREATED_AT - 이력 생성 시점(변경 시점)

GA_ASSET_ASSIGN_HISTORY 자산 지급 상태 정보
123 IDX - 인덱스
A-Z AST_KEY - 자산 번호
A-Z ASSIGN_TYPE - 자산 지급 이력 구분(지급/이관/반납)
A-Z ASSIGN_DATE - 이력 변경일
A-Z USER_ID - 자산 사용자(그룹웨어 ID)
A-Z USER_EMP_NO - 자산 사용자(사번)
A-Z USER_NM - 자산 사용자(이름)
A-Z DEPT_CD - 자산 사용 부서 코드
A-Z DEPT_NM - 자산 사용 부서명
A-Z PROJECT_CD - 자산 사용 프로젝트 코드
A-Z PROJECT_NM - 자산 사용 프로젝트명
A-Z MEMO - 비고
A-Z CREATE_USER - 등록자
🕒 CREATE_DATE - 등록일자

GA_ASSET_RNT_CTR 자산 렌탈 계약 정보
A-Z CTR_NO - 계약 번호
A-Z VENDOR_CD - 공급업체 코드
A-Z VENDOR_NM - 공급업체명
A-Z CTR_TITLE - 계약명
A-Z CTR_START_DATE - 계약 시작일
A-Z CTR_END_DATE - 계약 종료일
123 CTR_TOTAL_AMT - 계약 총 금액
123 CTR_MONTH_AMT - 월 렌탈 금액
A-Z CTR_STATUS - 계약 상태(계약중/일부계약중/종료)
A-Z CTR_MANAGER_ID - 계약 담당자 ID
A-Z CTR_MANAGER_EMP_NO - 계약 담당자 사번
A-Z CTR_MANAGER_NM - 담당자명
A-Z CTR_DEPT_CD - 담당 부서
A-Z CTR_DEPT_NM - 담당부서명
A-Z CTR_FILE_ID - 계약서 파일
A-Z MEMO - 비고
A-Z CREATE_USER - 등록자
🕒 CREATE_DATE - 등록일자
A-Z UPDATE_USER - 변경자
🕒 UPDATE_DATE - 변경일자

GA_ASSET
A-Z AST_KEY - 관리번호(자산 고유 식별자)
A-Z AST_GAIN_TYPE - 자산 취득 구분
A-Z MODEL_NM - 모델명
A-Z AST_TYPE - 자산 구분(유형/무형/토지/건물/사용권)
A-Z AST_CATEGORY - PC 관련 코드(모니터/데스크탑/노트북/빔프로젝터 등)
A-Z PRODUCT_NM - 품명
A-Z SERIAL_NO - 시리얼 넘버
A-Z SPEC_CPU - 사양: CPU
A-Z SPEC_RAM - 사양: RAM(메모리)
A-Z SPEC_SSD - 사양: SSD
A-Z SPEC_SIZE - 사양: 모니터 사이즈
A-Z MANUFACTURER - 제조사
A-Z BUY_DATE - 구매: 구매일
123 BUY_AMT - 구매: 구매금액
A-Z USER_ID - 자산 사용자(그룹웨어 ID)
A-Z USER_EMP_NO - 자산 사용자(사번)
A-Z USER_NM - 자산 사용자(이름)
A-Z DEPT_CD - 자산 사용 부서 코드
A-Z DEPT_NM - 자산 사용 부서명
A-Z PROJECT_CD - 자산 사용 프로젝트 코드
A-Z PROJECT_NM - 자산 사용 프로젝트명
A-Z AST_STATUS - 자산 상태(사용중/유휴 등)
A-Z MEMO - 비고
A-Z CREATE_USER - 등록자
🕒 CREATE_DATE - 등록일자
A-Z UPDATE_USER - 변경자
🕒 UPDATE_DATE - 변경일자

◆ AUTOMATION

자산 흐름이 비용 계산으로 자연스럽게 이어지도록

자산 렌탈 계약과 프로젝트 할당 기간을 연결해 프로젝트별 비용이 자동으로 집계되도록 구현했습니다.
임직원별 보유 자산 조회와 이동·퇴사 프로세스를 연동해 별도의 엑셀 대조 없이 시스템 안에서 현황을 확인할 수 있게 했습니다.

- ✓ 프로젝트별 렌탈 비용 자동 집계
- ✓ 임직원 보유 자산 실시간 조회
- ✓ 수기 대조와 중복 입력 제거

◆ OUTCOME

결과로 남은 것

자산 데이터 정합성 100%	분석부터 구축까지 6개월	전 과정 단독 수행 1인
--------------------	------------------	------------------

>>>>> 프로젝트별 자산 렌탈 비용 자동 집계 및 실시간 현황 관리 환경 구축

◆ SOLO MVP · 2026.08 — 운영 중

Poo Diary

식생활과 배변 패턴의 상관관계를 추적하는 PWA

- Next.js
- TanStack Query
- NestJS
- PostgreSQL
- Github Workflow
- GHCR
- ArgoCD
- Turborepo
- k3s

역할 | 기획 · 아키텍처 · 풀스택 · DevOps

규모 | 1명

배변 상태와 음식·컨디션 기록을 바탕으로 건강 패턴을 돌아볼 수 있는 귀여운 디자인의 모바일 중심 PWA입니다.

Github Actions와 Workflow를 활용하여 GitOps 빌드부터 배포까지 이어지는 파이프라인을 직접 구성하고, NHN Cloud에서 k3s(경량 쿠버네티스)를 설치하여 배포했습니다.



사내 시스템 화면으로, 보안을 위해 일부 정보와 식별 요소를 마스킹 처리했습니다.

◆ GOAL

건강 기록을 부담 없이 남기는 모바일 MVP

매일 반복되는 건강 신호를 간편하게 기록하고, 음식과 배변 상태의 연관성을 추적할 수 있도록 설계했습니다.
설치 장벽을 낮추기 위해 네이티브 앱 대신 PWA를 선택하고 모바일 화면에서 빠르게 기록하는 흐름에 집중했습니다.

- ✓ 모바일 중심 기록 경험
- ✓ PWA 설치 및 홈 화면 진입
- ✓ 일·주·월·전체 기간별 통계 제공 및 패턴 분석 기능

◆ VOC

지인 VOC를 기록 항목과 조건별 분석으로 연결하다

MVP를 지인들에게 공유한 뒤 ‘먹은 메뉴를 더 구체적으로 기록하고 싶다’, ‘생리·과식·수면 여부도 함께 확인하고 싶다’는 의견을 받았습니다.
피드백을 바탕으로 자유 입력 메뉴와 생리 일차, 충분한 수면 여부, 과식 여부를 기록 항목에 추가했습니다.
단순 항목 추가에 그치지 않고 통계 범위를 확장하여, 식단 및 생활 조건에 따른 배변 상태의 차이를 한눈에 비교할 수 있도록 구현했습니다.

- ✓ VOC 기반 구체 메뉴 입력 및 생활 조건 기록 추가
- ✓ 식품별 평균 브리스톨 스케일·통증 비율 및 빈출 메뉴 분석
- ✓ 생리 일차·수면·과식 여부에 따른 배변 상태 비교
- ✓ 최소 표본 수를 충족한 조건만 분석해 결과의 과도한 해석 방지

◆ IDENTITY MIGRATION

Device ID 사용자를 통합 계정으로 점진적으로 전환하다

초기 MVP는 별도 가입 없이 사용할 수 있도록 Device ID로 사용자를 식별했습니다.
진입 장벽은 낮았지만 기기가 바뀌면 기존 기록에 접근하기 어렵고, 여러 기기에서 같은 데이터를 이용할 수 없다는 한계가 있었습니다.

사이드 프로젝트 개수가 조금씩 늘어남에 따라 개인 도메인(Mercury Lab)의 KeyCloak 기반 통합 로그인 앱을 배포하고, Poo Diary에도 통합 로그인을 도입했습니다.
도입 후에도 기존 사용 방식을 즉시 중단하지 않고 Device ID 세션과 OIDC 계정 로그인을 병행하고 있습니다.
기존 사용자에게 통합 계정 생성 또는 로그인을 안내하고, 사용자가 연결을 선택한 경우에만 기존 기록을 Mercury Lab 계정에 연결하도록 구성했습니다.

계정 연결 상태와 데이터 이전 상태를 분리해 관리하고, 하나의 기기 계정이나 통합 계정이 중복 연결되지 않도록 제한했습니다.
연결에 실패하면 기존 기록과 Device ID 이용 흐름을 유지해 인증 전환 과정에서 데이터 접근이 끊기지 않도록 했습니다.

- ✓ Device ID 세션과 KeyCloak(OIDC) 통합 계정 로그인 병행 운영
- ✓ 사용자 동의 시 기존 기록을 통합 계정에 연결
- ✓ Fallback 시 기존 데이터와 이용 흐름 유지

Keycloak 기반 통합 인증 체계 구축 및 점진적 계정 마이그레이션 달성

◆ DELIVERY

NHN Cloud에서 배포 파이프라인을 직접 구성하다

NHN Cloud Compute 인스턴스에 k3s 환경을 구성하고, GitHub Actions에서 컨테이너 이미지를 빌드한 뒤 ArgoCD가 변경 사항을 배포하도록 연결했습니다.

Ingress 설정과 Cloudflare 커스텀 도메인을 연결하고, cert-manager를 도입해 Let's Encrypt SSL/TLS 인증서 자동 발급 및 갱신 체계를 구축했습니다. 이를 통해 코드 변경부터 HTTPS 보안 통신이 적용된 서비스 반영까지의 배포 흐름을 직접 구현했습니다.

- ✓ NHN Cloud Compute 내 k3s 인프라 환경 구축
- ✓ GitHub Actions를 활용한 컨테이너 이미지 빌드 및 GHCR 푸시
- ✓ ArgoCD 기반 GitOps 자동 배포 파이프라인 구축
- ✓ Ingress 및 cert-manager 활용 SSL/TLS 인증서 자동 발급·HTTPS 적용

NHN Cloud k3s 기반 GitOps 배포 파이프라인 및 cert-manager 인증서 자동화 구축

◆ SOLO PRODUCT · 2026.09 — 개발 및 테스트 중

공유 가계부

가계 구성원이 수입과 지출을 함께 관리할 수 있도록 설계한 웹 애플리케이션입니다.
'가계'를 워크스페이스 개념으로 도입하여, 워크스페이스를 자유롭게 추가할 수 있고 구성원을 초대할 수도 있습니다.
결제수단·카테고리별로 지출을 등록할 수 있고, 지출 리포트도 한 눈에 볼 수 있습니다.
현재는 직접 사용해보며 신용카드별 다양한 할인·적립 항목에 정확히 대응할 수 있는 방법을 찾고 있습니다.

◆ IDENTITY · 2026.09 — 통합 로그인 기본 구현

OIDC

개인 실험 프로젝트를 'Mercury Lab'이라는 이름으로 브랜딩하고,
개인 도메인에 배포한(할) 서비스들을 하나의 계정으로 이용할 수 있도록 Keycloak 기반 중앙 인증 체계를 설계했습니다.
Realm·Client·Scope·Audience·Role 구조와 Authorization Code + PKCE 기반 BFF 인증 흐름,
HttpOnly 쿠키 기반 토큰 갱신, 기존 계정의 안전한 연결과 단계적 전환 전략을 1차 구현했으며 운영 검증을 진행 중입니다.

◆ NPM LIBRARY · 2026.09 — 한국형 캘린더

캘린더 라이브러리

반응형 UI와 반복 일정·다중 캘린더 보기를 지원하는 TypeScript 기반 React 라이브러리
calendar-mercury-lab을 제작해 npm에 배포했습니다.
한국 공휴일 데이터 API 키를 전달하면 공휴일을 조회해 캘린더에 표시할 수 있으며,
공유 가계부의 일정 화면에 직접 적용해 사용하고 있습니다.

◆ Let's build something solid.

함께 풀어낼 문제가 있다면, 이야기를 들려주세요.

Email | uhjinkim9@gmail.com

GitHub | <https://github.com/uhjinkim9>

Portfolio | <https://portfolio.mercury-lab.uk>